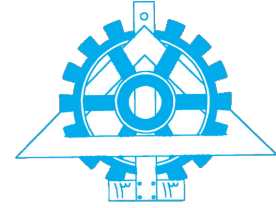




University of Tehran
College of Engineering
School of ECE



Machine Learning

Assignment #3

Student Name
Amirali Dehghani

Student ID
810102443

December 6, 2025

Contents

Abstract	1
1 Question 1	2
1.1 (Why is the dual formulation of SVM usually preferred when kernels are used? Provide two reasons.)	2
1.2 Explain the statement: "Only support vectors influence the final SVM classifier." Why do non-support vectors have zero dual coefficients? . .	3
1.3 Explain mathematically why the kernel trick allows SVMs to operate in infinite-dimensional spaces without computing the feature map explicitly.	3
1.4 Explain how the choice of kernel (linear, polynomial, RBF) affects the decision boundary, model capacity, and tendency to overfit.	4
2 Question 2: RBF kernel feature space intuition	5
3 Question 3: SVM dual symmetric 4-point example	6
4 Question 4: Harder Kernel-Trick Question (Explicit cubic feature map)	7
4.1	7
4.2	8
4.3	8
5 Question 5: The Mechanics of Splitting (ID3)	9
5.1 Root Entropy	9

5.2	Information Gain	9
5.3	Tree Construction	10
5.4	Comparison	10
6	Question 6: Optimization Geometry & Jensen's Inequality	11
6.1	Proof of Concavity	11
6.2	Jensen's Inequality	12
6.3	Optimization Trap	12
7	Question 7: The Failure of greedy Search (XOR)	13
7.1	The Greedy Trap	13
7.2	Existence	14
7.3	Inductive Bias	14
8	Question 8: From Data to Diagnosis: Diabetes Classification	
	Using SVM	15
8.1	Data Preprocessing and Exploration	15
8.2	Linear SVM Classification	16
8.3	Comparison of Different Kernels	17
8.4	Hyperparameter Tuning	19
8.5	Dimensionality Reduction and Visualization	20
8.6	Class Imbalance Analysis	23
9	Question 9: The QP-SVM Primal Challenge: Optimizing and	
	Visualizing the Soft-Margin Hyperplane SVM	24
9.1	Primal Optimization Problem	25
9.2	QP Formulation and Numerical Solution	26
9.3	Model Evaluation	26
9.4	Decision Boundary Visualization	26
10	Question 10: Decision Tree Classifier (ID3) Implementation	

and Analysis	27
10.1 Data Loading and High-Cardinality Trap	28
10.2 Entropy and Information Gain Engine	28
10.3 Handling Continuous Features (Temperature)	29
10.4 Recursive Tree Construction and Evaluation	30
10.5 Overfitting and Pruning Analysis	30
11 References	33

List of Figures

1	PCA projection of the training data onto the first two principal components.	21
2	Decision boundary learned by an RBF SVM on the PCA-reduced dataset.	22
3	Decision boundary of the primal QP-SVM plotted over the scaled training samples.	27
4	Tree Size vs. Test Accuracy under various pruning tolerances.	32

Abstract

This report presents a combined theoretical and practical study of key machine learning methods, with emphasis on Support Vector Machines (SVMs) and decision tree learning. On the theory side, we review the primal and dual formulations of SVMs, explain why the dual form is preferred in kernel-based models, and show how the kernel trick enables learning in high- or infinite-dimensional spaces without explicit feature maps. We compare linear, polynomial, and RBF kernels in terms of decision boundary complexity and overfitting, construct an explicit cubic feature map for a polynomial kernel, and analyze entropy and Information Gain, proving the concavity of entropy and the non-negativity of gain. The limitations of greedy, gain-based tree construction are illustrated using small synthetic datasets such as XOR.

On the experimental side, we apply SVMs to the Pima Indians Diabetes dataset to study the impact of feature scaling, kernel choice, hyperparameter tuning, PCA-based dimensionality reduction, and class imbalance handling. We further implement a primal soft-margin SVM as a quadratic program on the `Social_Network_Ads` dataset. Finally, we build an ID3 decision tree from scratch on an extended "Play Tennis" corpus, including handling of a high-cardinality identifier, continuous splits via optimal thresholds, and an overfitting analysis with label noise and reduced-error pruning, relating the observed behavior to tree complexity and generalization.

Question 1

1.1 (Why is the dual formulation of SVM usually preferred when kernels are used? Provide two reasons.)

The dual form of the SVM is generally preferred when kernel functions are used for the following reasons:

1. **Dependence on inner products:** The dual optimization problem depends only on the pairwise inner products $\langle x_i, x_j \rangle$. When kernels are applied, these inner products are replaced by the kernel function $K(x_i, x_j)$, allowing the SVM to operate in high-dimensional (even infinite-dimensional) feature spaces without explicitly computing the feature map. This makes the kernel trick feasible *only* in the dual formulation.
2. **Dimensionality advantage:** The primal problem has d parameters (the dimension of the feature space), which may be extremely large or infinite when kernels are used. In contrast, the dual problem has n variables (one α_i per training sample). Since $n \ll d$ in many real datasets, the dual problem is computationally much more efficient and remains solvable even when the feature space is infinite.

1.2 Explain the statement: “Only support vectors influence the final SVM classifier.” Why do non-support vectors have zero dual coefficients?

In the dual form of SVM, the decision function is:

$$f(x) = \sum_{i=1}^n \alpha_i y_i K(x_i, x) + b.$$

Only support vectors have non-zero coefficients α_i . The reasons are:

1. Points far from the decision boundary do not affect the optimal separating hyperplane. Since moving them does not change the margin, the optimization sets their α_i to zero.
2. Points on or near the margin **do** determine the hyperplane, so they obtain non-zero α_i . These points are called *support vectors* and are the only samples contributing to $f(x)$.

1.3 Explain mathematically why the kernel trick allows SVMs to operate in infinite-dimensional spaces without computing the feature map explicitly.

A kernel function satisfies

$$K(x, z) = \langle \phi(x), \phi(z) \rangle,$$

for some (possibly infinite-dimensional) feature map ϕ . In the dual SVM formulation, the classifier depends only on these inner products:

$$f(x) = \sum_i \alpha_i y_i K(x_i, x) + b.$$

Therefore, SVM never needs to compute $\phi(x)$ explicitly. Even if ϕ maps the data into an infinite-dimensional space (as with the RBF kernel), the optimization only requires evaluating $K(x_i, x_j)$, making high-dimensional or infinite-dimensional computation feasible without constructing the feature vectors.

1.4 Explain how the choice of kernel (linear, polynomial, RBF) affects the decision boundary, model capacity, and tendency to overfit.

- **Linear kernel:** Produces a linear decision boundary (a hyperplane). The model has low capacity and low risk of overfitting. Suitable when the data are approximately linearly separable.
- **Polynomial kernel:** Creates curved decision boundaries whose complexity increases with the polynomial degree. Higher degrees increase model capacity and may lead to overfitting.
- **RBF (Gaussian) kernel:** Generates highly flexible and smooth nonlinear boundaries. The model has very high capacity and can overfit if the kernel width σ is too small. Often performs best when tuned properly.

Question 2: RBF kernel feature space intuition

The RBF kernel is defined as:

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right).$$

Using the Taylor series expansion of the exponential function,

$$e^t = \sum_{k=0}^{\infty} \frac{t^k}{k!},$$

the kernel can be written as an infinite sum of polynomial terms in $\langle x, z \rangle$. This representation corresponds to the inner product of two vectors in an infinite-dimensional feature space:

$$K(x, z) = \langle \phi(x), \phi(z) \rangle,$$

where ϕ contains infinitely many components.

Thus, the RBF kernel implicitly performs computation in an infinite-dimensional Hilbert space without explicitly constructing the feature map.

Question 3: SVM dual symmetric 4-point example

Dual solution

$$\begin{aligned}x_1 &= (1, 1)^\top, \quad y_1 = +1, \quad x_2 = (1, -1)^\top, \quad y_2 = +1, \\x_3 &= (-1, 1)^\top, \quad y_3 = -1, \quad x_4 = (-1, -1)^\top, \quad y_4 = -1\end{aligned}$$

The dual problem is

$$\max_{\alpha} \sum_{i=1}^4 \alpha_i - \frac{1}{2} \sum_{i,j=1}^4 \alpha_i \alpha_j y_i y_j x_i^\top x_j, \quad \text{where } \sum_{i=1}^4 \alpha_i y_i = 0, \quad \alpha_i \geq 0$$

$$\xrightarrow{\text{Symmetry}} \alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \alpha$$

$$\rightarrow w = \sum_{i=1}^4 \alpha_i y_i x_i = \alpha (y_1 x_1 + y_2 x_2 + y_3 x_3 + y_4 x_4) = \alpha (4, 0)^\top$$

$$\Rightarrow \|w\|^2 = 16\alpha^2 \Rightarrow L_D(\alpha) = 4\alpha - \frac{1}{2}\|w\|^2 = 4\alpha - 8\alpha^2$$

Maximizing $L_D(\alpha)$ over $\alpha \geq 0$ gives

$$\begin{aligned}\frac{dL_D}{d\alpha} &= 4 - 16\alpha = 0 \quad \Rightarrow \quad \alpha = \frac{1}{4}. \\ \Rightarrow \quad &\boxed{\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \frac{1}{4}}\end{aligned}$$

Primal parameters w and b

$$w = \sum_{i=1}^4 \alpha_i y_i x_i \xrightarrow{\alpha=\frac{1}{4}} w = \frac{1}{4}(4, 0)^\top = (1, 0)^\top$$

To find b , we use any support vector. For example, for $x_1 = (1, 1)$ with $y_1 = +1$, on the margin we have

$$\begin{aligned} y_1(w^\top x_1 + b) = 1 &\Rightarrow (1)(1 \cdot 1 + 0 \cdot 1 + b) = 1 \Rightarrow 1 + b = 1 \Rightarrow b = 0 \\ &\implies \boxed{w = (1, 0)^\top, \quad b = 0} \end{aligned}$$

and the decision boundary is

$$w^\top x + b = 0 \iff x_1 = 0.$$

Question 4: Harder Kernel-Trick Question (Explicit cubic feature map)

4.1

$$K(x, z) = (1 + x^\top z)^3 = (1 + x_1 z_1 + x_2 z_2)^3.$$

Let $a = 1$, $b = x_1 z_1$ and $c = x_2 z_2$. Then

$$\begin{aligned} (a + b + c)^3 &= a^3 + 3a^2b + 3a^2c + 3ab^2 + 6abc + 3ac^2 \\ &\quad + b^3 + 3b^2c + 3bc^2 + c^3. \end{aligned}$$

Substituting back a, b, c gives

$$\begin{aligned} (1 + x_1 z_1 + x_2 z_2)^3 &= 1 + 3x_1 z_1 + 3x_2 z_2 \\ &\quad + 3x_1^2 z_1^2 + 6x_1 x_2 z_1 z_2 + 3x_2^2 z_2^2 \\ &\quad + x_1^3 z_1^3 + 3x_1^2 x_2 z_1^2 z_2 + 3x_1 x_2^2 z_1 z_2^2 + x_2^3 z_2^3. \end{aligned}$$

4.2

$$\phi(x) = \begin{pmatrix} 1 \\ \sqrt{3} x_1 \\ \sqrt{3} x_2 \\ \sqrt{3} x_1^2 \\ \sqrt{6} x_1 x_2 \\ \sqrt{3} x_2^2 \\ x_1^3 \\ \sqrt{3} x_1^2 x_2 \\ \sqrt{3} x_1 x_2^2 \\ x_2^3 \end{pmatrix}.$$

4.3

To verify correctness of the feature map, observe that the polynomial kernel $(K(x, z) = (1 + x^\top z)^3 = (1 + x_1 z_1 + x_2 z_2)^3)$ expands into a sum of 10 monomials of the form:

$$c x_1^a x_2^b z_1^a z_2^b, \quad a, b \geq 0, \quad a + b \leq 3,$$

where c is the corresponding multinomial coefficient.

For each monomial with coefficient c , the feature map contains the component $\sqrt{c} x_1^a x_2^b$, and likewise $\phi(z)$ contains $\sqrt{c} z_1^a z_2^b$

Thus the dot product becomes

$$\phi(x)^\top \phi(z) = \sum_{a+b \leq 3} (\sqrt{c} x_1^a x_2^b) (\sqrt{c} z_1^a z_2^b) = \sum_{a+b \leq 3} c x_1^a x_2^b z_1^a z_2^b,$$

which reconstructs exactly the expanded polynomial of $(1 + x^\top z)^3$.
Therefore,

$$K(x, z) = \phi(x)^\top \phi(z)$$

Question 5: The Mechanics of Splitting (ID3)

5.1 Root Entropy

$$\begin{aligned} Y &= \{0, 0, 1, 0, 1, 1\} \\ \Rightarrow p_0 &= \frac{3}{6} = \frac{1}{2}, \quad p_1 = \frac{3}{6} = \frac{1}{2} \\ \Rightarrow H(Y) &= -p_0 \log_2 p_0 - p_1 \log_2 p_1 = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1. \end{aligned}$$

5.2 Information Gain

- $X_1 = A$: labels $\{0, 0, 1\}$

$$\Rightarrow H(Y \mid X_1 = A) = -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} \approx 0.918.$$

- $X_1 = B$: labels $\{0, 1, 1\}$

$$\Rightarrow H(Y \mid X_1 = B) \approx 0.918.$$

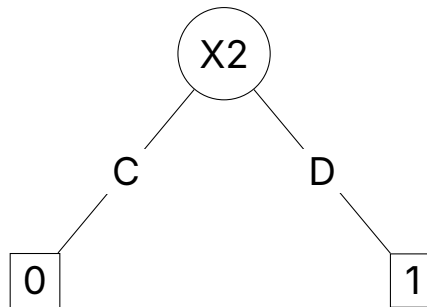
$$\begin{aligned}\text{Gain}(S, X_1) &= H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v) \\ &= 1 - \left(\frac{3}{6} H_A + \frac{3}{6} H_B \right) \approx 1 - 0.918 \approx 0.082.\end{aligned}$$

- $X_2 = C$: labels $\{0, 0, 0\}$, so $H(Y \mid X_2 = C) = 0$.
- $X_2 = D$: labels $\{1, 1, 1\}$, so $H(Y \mid X_2 = D) = 0$.

$$\implies \text{Gain}(S, X_2) = H(S) - \left(\frac{3}{6} \cdot 0 + \frac{3}{6} \cdot 0 \right) = 1.$$

Since $\text{Gain}(S, X_2) > \text{Gain}(S, X_1)$, ID3 chooses X_2 as the root attribute.

5.3 Tree Construction



Each leaf is pure, and all training samples are classified correctly, so the tree achieves zero training error.

5.4 Comparison

ID3 builds trees in a top-down, greedy manner, choosing at each step the attribute with maximum information gain. A hypothetical bottom-up procedure would start from smaller

subtrees or leaves and merge them upward. In general, different construction strategies may produce different but *equivalent* trees (i.e., representing the same function on the training set), so the bottom-up tree is not guaranteed to be identical to the top-down ID3 tree.

Question 6: Optimization Geometry & Jensen's Inequality

6.1 Proof of Concavity

$$H(p) = -p \log_2 p - (1-p) \log_2 (1-p) \quad 0 < p < 1$$

$$\xrightarrow{\log_2 x = \frac{\ln x}{\ln 2}} H(p) = -\frac{1}{\ln 2} [p \ln p + (1-p) \ln(1-p)]$$

$$\xrightarrow{\frac{d}{dp}} H'(p) = \frac{\ln\left(\frac{1-p}{p}\right)}{\ln 2} \xrightarrow{\frac{d}{dp}} H''(p) = \frac{1}{p(p-1) \ln 2}$$

$$0 < p < 1 \Rightarrow p(p-1) < 0$$

$$\Rightarrow \boxed{H''(p) < 0 \quad 0 < p < 1}$$

so $H(p)$ is strictly **concave** on $(0, 1)$.

6.2 Jensen's Inequality

$$\begin{aligned} S &= \bigcup_{i=1}^k S_i, & \lambda_i &= \frac{|S_i|}{|S|}, & \sum_i \lambda_i &= 1 \\ p_i &= \Pr(Y = 1 \mid S_i), & p &= \Pr(Y = 1 \mid S) = \sum_i \lambda_i p_i \\ H(S) &= H(p), & H(S_i) &= H(p_i) \end{aligned}$$

Information Gain:

$$\begin{aligned} \text{Gain}(S, X) &= H(S) - \sum_i \lambda_i H(S_i) \\ &= H\left(\sum_i \lambda_i p_i\right) - \sum_i \lambda_i H(p_i) \end{aligned}$$

Since $H(p)$ is strictly concave on $(0, 1)$, Jensen's inequality gives:

$$H\left(\sum_i \lambda_i p_i\right) \geq \sum_i \lambda_i H(p_i)$$

Thus,

$$\boxed{\text{Gain}(S, X) \geq 0}$$

6.3 Optimization Trap

ID3 builds decision trees in a top-down, greedy fashion, choosing at each node the attribute with the highest immediate information gain. This hill-climbing strategy can get stuck in a local optimum.

For example, suppose attributes A and B are available at the root. Splitting on A yields slightly higher immediate gain than splitting on B , so ID3 chooses A at the root. However, the best global tree might require splitting on B at the root and then on another attribute (e.g. C) at the next level, leading to a much lower overall training error. Because ID3 never revisits earlier choices and only considers local gain, it may select A and miss the better tree rooted at B , thus converging to a local rather than a global optimum.

Question 7: The Failure of greedy Search (XOR)

7.1 The Greedy Trap

The dataset is:

$$(0, 0) \rightarrow 0, \quad (0, 1) \rightarrow 1, \quad (1, 0) \rightarrow 1, \quad (1, 1) \rightarrow 0.$$

The class distribution is balanced, so

$$H(S) = 1.$$

Gain of X_1

$$S_{X_1=0} : \{0, 1\} \Rightarrow H = 1, \quad S_{X_1=1} : \{1, 0\} \Rightarrow H = 1$$

$$\text{Gain}(S, X_1) = 1 - \left(\frac{2}{4} \cdot 1 + \frac{2}{4} \cdot 1 \right) = 0.$$

Gain of X_2

$$S_{X_2=0} : \{0, 1\} \Rightarrow H = 1, \quad S_{X_2=1} : \{1, 0\} \Rightarrow H = 1$$

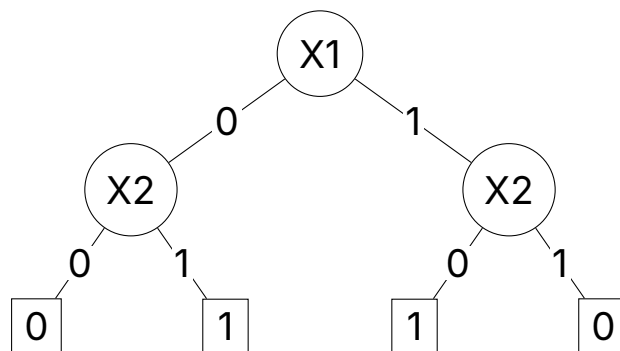
$$\text{Gain}(S, X_2) = 1 - \left(\frac{2}{4} \cdot 1 + \frac{2}{4} \cdot 1 \right) = 0.$$

Thus:

$$\text{Gain}(S, X_1) = \text{Gain}(S, X_2) = 0$$

ID3 cannot choose a root attribute (all splits provide zero reduction in entropy).

7.2 Existence



7.3 Inductive Bias

(a) Preference Bias

A preference bias favors certain hypotheses over others, without removing any hypothesis from the hypothesis space. The learner searches the full space but “prefers” some hypotheses.

(b) Why XOR illustrates preference bias

Although XOR is representable by a decision tree (depth 2), ID3 fails to discover it because both X_1 and X_2 yield zero information gain at the root. ID3 prefers trees whose *first split* produces high gain, and

therefore does not explore deeper trees with low-gain initial splits.
Thus:

ID3 does not restrict the hypothesis space; it prefers high-gain roots, leading to a preference bias.

Question 8: From Data to Diagnosis: Diabetes Classification Using SVM

8.1 Data Preprocessing and Exploration

Pima Indians Diabetes Dataset

Why is feature scaling important for SVM?

Support Vector Machines are sensitive to the scale of the input features because their decision function is based on distances and inner products in the feature space. If one feature has a much larger numeric range than the others, it will dominate the dot products and the geometry of the margin. This can lead to a distorted decision boundary and poor generalization.

By applying feature scaling (e.g. standardization to zero mean and unit variance), all features are brought to a comparable scale. This makes the SVM optimization more stable, allows the margin to reflect the true structure of the data, and typically improves both convergence and classification performance.

8.2 Linear SVM Classification

(a) Linear SVM without feature scaling

A linear SVM classifier was trained on the raw (unnormalized) features using `LinearSVC` with `random_state = 443`. The evaluation on the test set produced:

$$\text{Accuracy} = 0.7273$$

$$\text{Confusion Matrix} = \begin{pmatrix} 132 & 18 \\ 45 & 36 \end{pmatrix}$$

(b) Linear SVM with feature scaling

The same classifier was trained again after standardizing all input features via `StandardScaler`. Test performance:

$$\text{Accuracy} = 0.7316$$

$$\text{Confusion Matrix} = \begin{pmatrix} 132 & 18 \\ 44 & 37 \end{pmatrix}$$

(c) Comparing the performance before and after feature scaling

Feature scaling led to a slight improvement in performance:

$$0.7273 \rightarrow 0.7316.$$

True positives increased ($36 \rightarrow 37$) and false negatives decreased ($45 \rightarrow 44$), indicating a marginally better decision boundary.

Normalization affects linear SVM because the model depends on inner products and distances in feature space. When features have different numeric scales, the larger-scale features dominate the geometry of the margin. After scaling, all features contribute more evenly, producing a more stable and better-aligned hyperplane. Although the improvement here is modest, it is consistent with the expected behavior of linear SVMs under proper normalization.

8.3 Comparison of Different Kernels

In this section, SVM classifiers were trained using three different kernels: linear, RBF, and polynomial. All models were trained on normalized features (StandardScaler), as required for stable kernel SVM performance.

Linear Kernel

$$\text{Accuracy} = 0.7316, \quad \text{Precision} = 0.6792$$

$$\text{Recall} = 0.4444, \quad \text{F1-score} = 0.5373$$

$$\text{Confusion Matrix} = \begin{pmatrix} 133 & 17 \\ 45 & 36 \end{pmatrix}$$

RBF Kernel

$$\text{Accuracy} = 0.7662, \quad \text{Precision} = 0.7368$$

$$\text{Recall} = 0.5185, \quad \text{F1-score} = 0.6087$$

$$\text{Confusion Matrix} = \begin{pmatrix} 135 & 15 \\ 39 & 42 \end{pmatrix}$$

Polynomial Kernel

$$\text{Accuracy} = 0.7013, \quad \text{Precision} = 0.6429$$

$$\text{Recall} = 0.3333, \quad \text{F1-score} = 0.4390$$

$$\text{Confusion Matrix} = \begin{pmatrix} 135 & 15 \\ 54 & 27 \end{pmatrix}$$

Comparison Table

Kernel	Accuracy	Precision	Recall	F1-score
Linear	0.7316	0.6792	0.4444	0.5373
RBF	0.7662	0.7368	0.5185	0.6087
Polynomial	0.7013	0.6429	0.3333	0.4390

Discussion

Among the three kernels, the RBF kernel achieves the best performance across all evaluation metrics. Its accuracy, precision, recall, and F1-score are consistently higher than those of the linear and polynomial kernels.

This behavior is expected because the Pima Diabetes dataset exhibits nonlinear structure, and the RBF kernel provides a flexible nonlinear decision boundary. The linear kernel is limited to a single hyperplane, while the polynomial kernel of degree 3 overfits some regions and underfits others. The RBF kernel achieves the best

bias–variance tradeoff for this dataset.

8.4 Hyperparameter Tuning

We tuned the hyperparameters of an RBF SVM classifier using `GridSearchCV`. The search grid was defined as

$$C \in \{0.1, 1, 10, 100\}, \quad \gamma \in \{0.001, 0.01, 0.1, 1\}.$$

1) Optimal hyperparameters

Grid search with 5-fold cross-validation selected:

$$C^* = 100, \quad \gamma^* = 0.001.$$

The best mean cross-validation accuracy obtained during the search was

$$\text{Best CV Accuracy} = 0.7895.$$

2) Test performance of the tuned model

The optimal model was then evaluated on the held-out test set:

$$\text{Test Accuracy} = 0.7316.$$

The corresponding confusion matrix is:

$$\begin{pmatrix} 133 & 17 \\ 45 & 36 \end{pmatrix}.$$

3) Discussion

The grid search favored a large value of C and a small value of γ , which corresponds to a wider RBF kernel and a stricter margin

penalty. This combination helps prevent overfitting while still maintaining a flexible, nonlinear decision boundary.

Although the cross-validation accuracy was relatively high (0.7895), the final test accuracy is comparable to the untuned RBF model. This behavior is expected for the Pima dataset, which is noisy and moderately imbalanced.

Conceptual Question: Effect of Very Large Values of C

In an SVM with a soft margin, the parameter C controls the penalty for misclassifying training samples. When C is very large, the optimizer is strongly discouraged from allowing any classification errors. As a result, the model attempts to correctly classify every training point, even if this requires a highly curved or low-margin decision boundary.

Consequently, a very large C produces:

- a narrower margin,
- a more complex decision boundary,
- high sensitivity to noise and outliers,
- increased risk of overfitting.

Thus, extremely large values of C typically lead to poor generalization performance.

8.5 Dimensionality Reduction and Visualization

PCA to Two Dimensions

Principal Component Analysis (PCA) was applied to the standardized dataset in order to reduce the dimensionality from 8 features to 2

principal components. The PCA projection of the training data is shown in Figure 1.

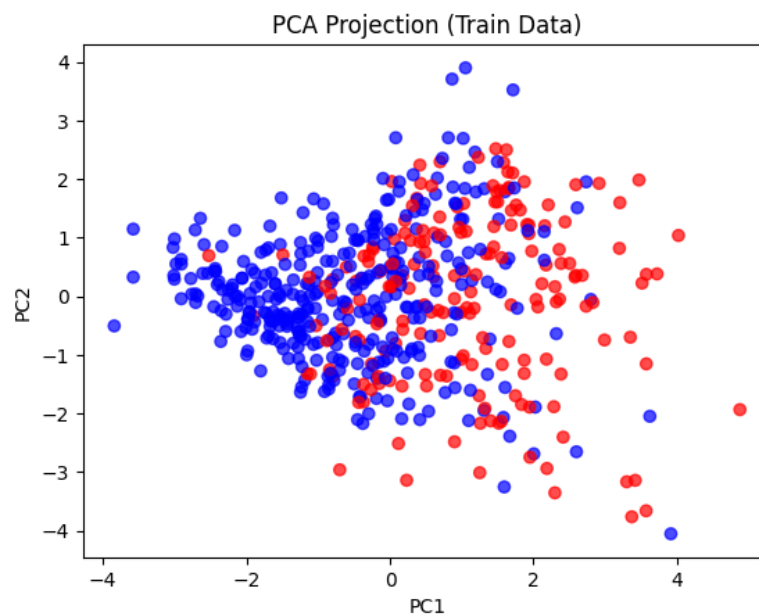


Figure 1: PCA projection of the training data onto the first two principal components.

Visualization of the Data in 2D

The two-dimensional PCA embedding reveals substantial overlap between the two classes, indicating that a linear separation in this reduced space is unlikely. This motivates the use of a nonlinear classifier such as an RBF SVM.

Training an RBF SVM on PCA-Reduced Data

An SVM classifier with an RBF kernel was trained on the two-dimensional PCA features. The model was then evaluated on the PCA-transformed test set.

$$\text{Accuracy (RBF SVM on PCA data)} = 0.684$$

The resulting decision boundary in the PCA space is shown in Figure 2.

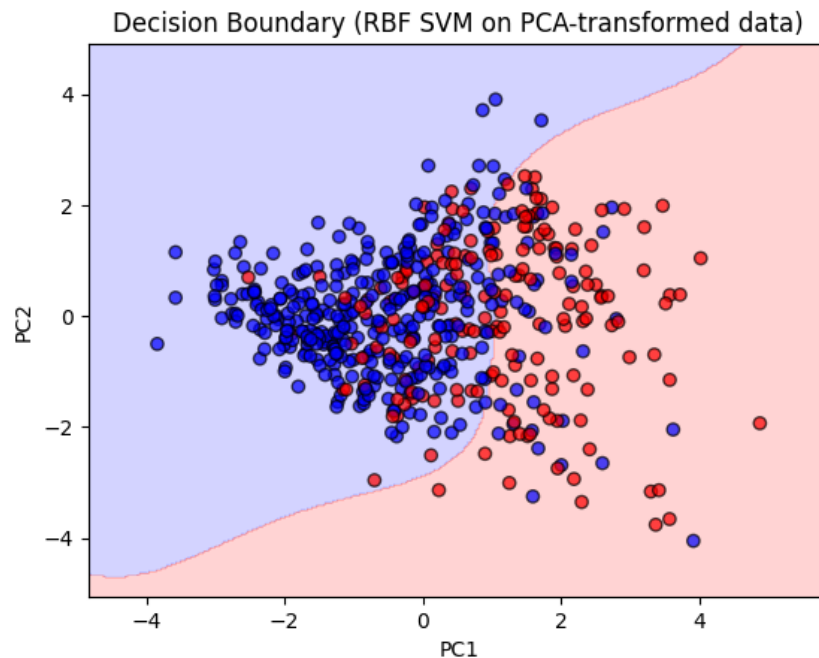


Figure 2: Decision boundary learned by an RBF SVM on the PCA-reduced dataset.

Discussion

Dimensionality reduction significantly affected the classification accuracy. While the original 8-dimensional models achieved accuracies in the range 0.73–0.77, the PCA-reduced 2D representation resulted in a noticeably lower accuracy of 0.684.

This drop in performance is expected because PCA preserves directions of maximum variance rather than directions that are most discriminative for classification. Important class-separating information may lie in lower-variance directions, and reducing the dimensionality to only two components removes much of this information. Consequently, the RBF SVM is unable to construct a sufficiently accurate decision boundary in the reduced space.

8.6 Class Imbalance Analysis

Class distribution

The dataset contains the following number of samples per class:

Class 0: 500, Class 1: 268.

Since class 1 is significantly smaller than class 0, the dataset is imbalanced.

Standard RBF SVM

A standard RBF SVM was trained on the normalized dataset. Its performance on the test set is:

$$\text{Accuracy} = 0.7662, \quad \text{Recall}_{(1)} = 0.5185$$

$$\text{Confusion Matrix} = \begin{pmatrix} 135 & 15 \\ 39 & 42 \end{pmatrix}$$

The recall for class 1 is relatively low, indicating that many minority-class samples are misclassified as class 0.

RBF SVM with `class_weight = "balanced"`

We retrained the classifier with automatically reweighted classes:

$$\text{Accuracy} = 0.7446, \quad \text{Recall}_{(1)} = 0.6667$$

$$\text{Confusion Matrix} = \begin{pmatrix} 118 & 32 \\ 27 & 54 \end{pmatrix}$$

Comparison and Discussion

Using class balancing substantially improved the recall of the minority class:

$$0.5185 \rightarrow 0.6667.$$

This happened because the balanced SVM assigns a higher penalty to misclassifying class 1 samples, reducing false negatives ($39 \rightarrow 27$) and increasing true positives ($42 \rightarrow 54$).

Although the overall accuracy decreased slightly, the classifier became significantly better at detecting the minority class, which is the desired effect when dealing with imbalanced datasets.

Conclusion: Yes, class balancing improved the performance for the minority class.

Question 9: The QP-SVM Primal Challenge: Optimizing and Visualizing the Soft-Margin Hyperplane SVM

In this question, we implement the primal formulation of a soft-margin linear Support Vector Machine (SVM) and solve it as a Quadratic Programming (QP) problem using `cvxpy`. Following the problem statement, we work with the dataset `Social_Network_Ads.csv` and restrict the features to `Age` and

EstimatedSalary. The target labels are mapped from $\{0, 1\}$ to $\{-1, 1\}$.

9.1 Primal Optimization Problem

The soft-margin primal SVM problem is defined as:

$$\min_{w, b, \xi} \quad \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i, \quad (1)$$

$$\text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n. \quad (2)$$

Here:

- $w \in \mathbb{R}^2$ is the weight vector,
- $b \in \mathbb{R}$ is the bias term,
- ξ_i are slack variables that allow constraint violations,
- $C > 0$ controls the trade-off between having a large margin and minimizing misclassification errors.

Effect of the Regularization Parameter C

The parameter C directly influences how the optimization balances the margin with the penalty on misclassifications. A large value of C heavily penalizes violations:

$C \uparrow \Rightarrow$ model strongly prioritizes reducing classification errors over maximizing the margin.

Thus, the classifier becomes less tolerant of slack and may overfit by forcing perfect separation.

9.2 QP Formulation and Numerical Solution

After preprocessing (train–test split and feature scaling), the optimization variables are defined as:

$$w \in \mathbb{R}^2, \quad b \in \mathbb{R}, \quad \xi \in \mathbb{R}_{\geq 0}^n.$$

Solving the QP via `cvxpy` yields the optimal parameters:

$$w^* = \begin{bmatrix} 1.5526 \\ 0.9297 \end{bmatrix}, \quad b^* = -0.7343.$$

9.3 Model Evaluation

Predictions are computed using the decision function:

$$\hat{y} = \text{sign}(w^{*\top} x + b^*).$$

The resulting accuracy on the test set is:

$\text{Accuracy} = 0.875.$

9.4 Decision Boundary Visualization

The separating hyperplane induced by the optimized parameters is given by:

$$w_1^* x_1 + w_2^* x_2 + b^* = 0.$$

This line is plotted on the scaled training data to illustrate the linear separation learned by the model.

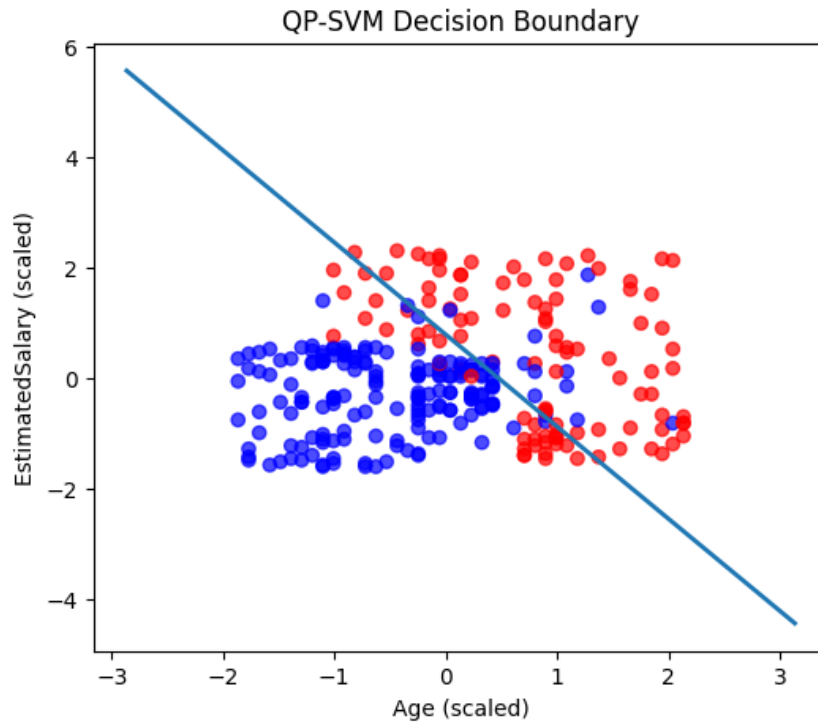


Figure 3: Decision boundary of the primal QP-SVM plotted over the scaled training samples.

Question 10: Decision Tree Classifier (ID3) Implementation and Analysis

In this project, we implement a full Decision Tree classifier from scratch, based on the ID3 algorithm, capable of handling both discrete and continuous features. The model is trained and evaluated on the Extended `jeeves_tennis.csv` dataset. Following the problem statement, we analyze the effect of high-cardinality features, compute Information Gain for candidate splits, extend ID3 to continuous attributes, evaluate the classifier on a held-out set, and finally investigate overfitting and pruning.

10.1 Data Loading and High-Cardinality Trap

The dataset contains the following attributes:

{Day_ID, Outlook, Temperature, Humidity, Wind, PlayTennis}.

The column Day_ID is a unique identifier. To evaluate its usefulness, we compute the entropy of the target label:

$$H(S) \approx 0.98145.$$

The Information Gain of splitting on Day_ID is:

$$\text{IG}(\text{Day_ID}) = 0.98145.$$

This value is equal to the root entropy because splitting on a near-unique identifier creates partitions of size 1, each of which is perfectly pure. Mathematically:

$$H(S_v) = 0 \quad \Rightarrow \quad \text{IG} = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v) = H(S).$$

Although the IG is numerically large, it is *useless for generalization*, because no meaningful pattern is learned. Therefore, we remove this feature before training:

$$S \leftarrow S \setminus \{\text{Day_ID}\}.$$

10.2 Entropy and Information Gain Engine

We implement the standard entropy function

$$H(S) = - \sum_{c \in \{Yes, No\}} p_c \log_2 p_c,$$

and the Information Gain:

$$\text{IG}(S, A) = H(S) - \sum_{v \in A} \frac{|S_v|}{|S|} H(S_v).$$

After removing Day_ID, we compute IG for the discrete attributes:

$$\text{IG}(\text{Outlook}) = 0.34307,$$

$$\text{IG}(\text{Humidity}) = 0.05389,$$

$$\text{IG}(\text{Wind}) = 0.03823.$$

Thus, the attribute selected for the root split is: Outlook.

10.3 Handling Continuous Features (Temperature)

Standard ID3 does not support real-valued attributes. To extend it, we search for a threshold t such that splitting into

$$\text{Temperature} \leq t \quad \text{vs.} \quad \text{Temperature} > t$$

maximizes the Information Gain.

Candidate thresholds are midpoints between adjacent samples with different labels:

$$t_i = \frac{x_i + x_{i+1}}{2}.$$

We evaluate the IG for all candidates and obtain:

$$\text{}t^* = 30.05\text{}, \quad \text{IG}(t^*) = 0.02392.$$

This threshold is used whenever Temperature is chosen for splitting within the recursive tree.

10.4 Recursive Tree Construction and Evaluation

We implement a full ID3 classifier with the following capabilities:

- recursive splitting on discrete attributes,
- threshold-based binary splitting on continuous attributes,
- stopping rules for pure nodes or empty feature sets,
- a prediction function that traverses the learned tree.

Using an 80/20 train–test split, evaluation yields:

$$\begin{aligned}\text{Accuracy}_{\text{test}} &= 0.93, \\ \text{Precision}(\text{Yes}) &= 0.9032, \\ \text{Recall}(\text{Yes}) &= 0.9824.\end{aligned}$$

The high recall indicates that the model tends to correctly identify almost all *positive* instances, though at the cost of precision.

10.5 Overfitting and Pruning Analysis

Decision Trees are prone to overfitting due to recursive partitioning until purity is achieved. We analyze their behavior under clean and noisy conditions and study the effectiveness of post-pruning.

(a) Depth Analysis on Clean Data

Training ID3 on the clean dataset yields:

$$\text{Depth}_{\text{clean}} = 12, \quad \text{LeafCount}_{\text{clean}} = 37.$$

The depth is relatively large for such a small dataset, indicating memorization tendencies.

(b) Noise Simulation (10% Label Flipping)

We randomly flip the labels of 10% of the training samples and retrain the model. The resulting tree becomes significantly larger:

$$\text{Depth}_{\text{noisy}} = 13, \quad \text{LeafCount}_{\text{noisy}} = 100.$$

The model attempts to fit noise by growing additional branches, confirming that ID3 is highly sensitive to noisy labels.

(c) Post-Pruning Experiment (Reduced Error Pruning)

We implement Reduced Error Pruning using a validation set and evaluate multiple pruning tolerances:

$$\tau \in \{0, 0.005, 0.01, 0.02, 0.05\}.$$

A subtree T is replaced by a leaf L when:

$$\text{Acc}_{\text{val}}(L) \geq \text{Acc}_{\text{val}}(T) - \tau.$$

Surprisingly, pruning does not change the size of the tree nor improve test performance:

τ	Tree Size (nodes)	Test Accuracy
0.0	13	0.91
0.005	13	0.91
0.01	13	0.91
0.02	13	0.91
0.05	13	0.91

This behavior is expected due to:

- the small size of the dataset,

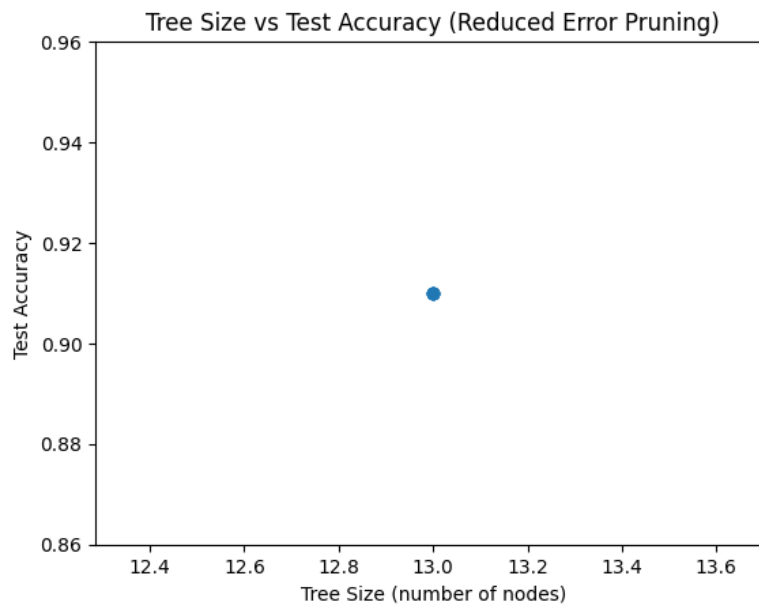


Figure 4: Tree Size vs. Test Accuracy under various pruning tolerances.

- limited label noise,
- insufficient validation samples to justify subtree removal.

Thus, pruning does not meaningfully simplify the tree or improve generalization.

References

1. Abolghasemi Dehaqani, M. (Fall 2025). *Machine Learning Basics – Support Vector Machines Slides*. University of Tehran. (Primary reference for SVM kernels.)
2. Abolghasemi Dehaqani, M. (Fall 2025). *Machine Learning Basics – Decision Trees Slides*. University of Tehran. (Primary reference for ID3, Entropy and Information Gain.)
3. Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press. (Reference for Lagrangian duality, KKT conditions, SVM primal/dual formulation.)
4. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. (Reference for kernel methods, support vector machines, feature maps, and PCA.)
5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer. (Reference for the geometry of SVMs, margin theory, and RBF kernels.)
6. Mitchell, T. (1997). *Machine Learning*. McGraw-Hill. (Reference for ID3, preference bias vs. restriction bias, and decision-tree learning.)
7. Scikit-Learn Documentation. Available at: <https://scikit-learn.org/>. (Used for SVM classifiers, PCA,

GridSearchCV, class_weight analysis, and performance metrics.)

8. Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization* (2nd ed.). Springer. (Reference for optimization geometry used in SVM derivations.)
9. Cover, T., & Thomas, J. (2006). *Elements of Information Theory*. Wiley. (Reference for entropy, concavity of $H(p)$, and Jensen's inequality.)